Servicio Nacional de Aprendizaje – SENA

Centro Agroempresarial y Desarrollo Pecuario del Huila – Garzón

Tecnólogo en Análisis y Desarrollo de Software (228118)

Informe técnico de plan de trabajo para construcción de software

Evidencia GA7-220501096-AA1-EV01

Proyecto: Tienda Deportes · Node.js, Express, React y MySQL

Aprendiz: [Nombre completo del aprendiz]

Ficha: [Número de ficha]

Instructor: [Nombre del instructor]

Garzón, Huila · Noviembre de 2026

Contenido

1. Introducción

2. Objetivos

    2.1 Objetivo general

    2.2 Objetivos específicos

3. Descripción del software

4. Plan de trabajo

    4.1 Metodología

    4.2 Módulos y entregables

    4.3 Equivalencia con las tecnologías de la guía

    4.4 Cronograma

5. Herramientas y tecnologías de desarrollo

6. Selección de herramientas de versionamiento

    6.1 Tipos de sistemas de control de versiones

    6.2 Plataformas para el repositorio remoto

    6.3 Herramientas seleccionadas

7. Estrategia de versionamiento

    7.1 Organización del repositorio

    7.2 Modelo de ramas

    7.3 Convención de mensajes de commit

    7.4 Versiones y etiquetas

    7.5 Archivos de configuración

    7.6 Flujo de trabajo diario

8. Integración continua

9. Buenas prácticas de seguridad

10. Conclusiones

11. Referencias

1. Introducción

Este informe define el plan de trabajo para construir el software de la Tienda Deportes durante la Guía de Aprendizaje 7, y selecciona las herramientas y tecnologías de versionamiento con las que se controlará el código. El plan parte de los artefactos elaborados en las guías anteriores: los requerimientos, la base de datos deportes (tablas Users, productos, pedido y detalle_pedido), los mockups de navegación y el front-end en HTML, CSS y JavaScript.

Un sistema de control de versiones registra cada cambio del código, permite volver a una versión anterior, trabajar en varias funcionalidades al mismo tiempo sin dañar lo que ya funciona y tener una copia de seguridad en la nube. Por eso se elige antes de escribir la primera línea de código.

2. Objetivos

2.1 Objetivo general

Definir el plan de trabajo para la construcción del software de la Tienda Deportes, especificando las herramientas y tecnologías de versionamiento que se utilizarán para controlar el código fuente del proyecto.

2.2 Objetivos específicos

- Interpretar los artefactos de diseño para identificar los módulos que se van a construir.
- Organizar las actividades de construcción en un cronograma por sprints.
- Seleccionar las herramientas de desarrollo según las tecnologías de la guía.
- Comparar sistemas y plataformas de control de versiones y justificar la selección.
- Definir la organización del repositorio, el modelo de ramas y las convenciones de commits.

3. Descripción del software

La Tienda Deportes vende balones, ropa y accesorios deportivos en Garzón (Huila). El software permitirá que los clientes se registren, consulten el catálogo con precio y stock, armen un carrito y registren pedidos, y que la tienda administre sus productos e inventario.

Código	Requerimiento funcional	Tablas
RF-01	Registrar una cuenta de cliente	Users
RF-02	Iniciar sesión con correo y contraseña	Users
RF-03	Consultar el catálogo con precio y stock	productos

Código	Requerimiento funcional	Tablas
RF-04	Buscar y filtrar productos	productos
RF-05	Registrar pedidos desde el carrito	pedido, detalle_pedido
RF-06	Consultar el historial de pedidos	pedido, detalle_pedido
RF-07	Administrar productos (crear, consultar, actualizar, eliminar)	productos

Artefactos de entrada del plan: diagrama de clases y casos de uso (Fase de planeación), modelo y script de la base de datos deportes (Guía 6, AA1 y AA2), mockups de navegación (AA3-EV02), interfaz gráfica (AA3-EV03) y diseño front-end (AA4-EV03).

4. Plan de trabajo

4.1 Metodología

Se trabajará con **Scrum** adaptado a un aprendiz: sprints de dos semanas, un tablero en GitHub Projects con las columnas Por hacer, En progreso, En revisión y Hecho, y una revisión al final de cada sprint para entregar la evidencia correspondiente. Cada historia de usuario se convierte en un issue de GitHub y se cierra con un pull request.

4.2 Módulos y entregables

Actividad	Módulo que se construye	Tecnología	Evidencias
AA1 · 40 h	Repositorio, herramientas y estándares	Git, GitHub	EV01 a EV05
AA2 · 96 h	CRUD de productos con MySQL; formularios con GET y POST	Node.js, Express, mysql2, EJS	EV01, EV02
AA3 · 122 h	Módulos de usuarios, catálogo y pedidos organizados en capas	Express	EV01, EV02
AA4 · 96 h	Front-end de la tienda	React	EV01 a EV03
AA5 · 144 h	Servicios web de autenticación y del proyecto	API REST con Express, JWT, Postman	EV01 a EV04

4.3 Equivalencia con las tecnologías de la guía

La guía propone tecnologías de Java para la AA2 y la AA3. El proyecto usará **Node.js con Express** en todo el back-end porque es el mismo lenguaje del front-end (JavaScript), lo que permite concentrarse en un solo lenguaje, compartir validaciones entre cliente y servidor y usar npm como único gestor de paquetes. Cada tecnología de la guía tiene su equivalente:

Tecnología de la guía	Equivalente en el proyecto	Qué hace
JDBC	mysql2 (consultas parametrizadas)	Conectar con MySQL y ejecutar el CRUD
Servlets	Rutas y controladores de Express	Recibir peticiones GET y POST y responder
JSP	Plantillas EJS	Generar páginas HTML con datos en el servidor
Framework Java (Spring Boot)	Express organizado en capas	Estructurar los módulos del software
Maven	npm	Gestionar dependencias y scripts
Tomcat	Servidor HTTP de Node.js	Ejecutar la aplicación

4.4 Cronograma

Sprint	Semanas	Actividades	Resultado del sprint
S1	1-2	Plan de trabajo, estándares, instalación de Git y GitHub	Repositorio creado y configurado (v0.1.0)
S2	3-4	Conexión a MySQL con mysql2 y CRUD de productos	Módulo de datos con pruebas manuales
S3	5-6	Formularios con EJS y métodos GET y POST	Módulo web (v0.2.0)
S4	7-8	Express en capas: usuarios y catálogo	Módulos organizados por capas
S5	9-10	Pedidos, validaciones y documento de pruebas	Módulos probados (v0.3.0)
S6	11-12	Taller y definición de componentes React	Documentos AA4-EV01 y EV02
S7	13-14	Front-end en React conectado a la API	Front-end (v0.4.0)
S8	15-16	API de registro e inicio de sesión y pruebas en Postman	AA5-EV01 y EV02
S9-S10	17-20	API del proyecto, documentación y pruebas	Versión 1.0.0

Nota: El número de semanas es una referencia: ajústalo al calendario que fije el instructor para la ficha.

5. Herramientas y tecnologías de desarrollo

Categoría	Herramienta	Uso en el proyecto
Entorno de ejecución	Node.js 24 (LTS) y npm	Ejecutar el back-end y gestionar dependencias
Framework back-end	Express 5	Rutas, controladores y API REST (AA2, AA3 y AA5)
Acceso a datos	mysql2	Conexión a MySQL con consultas parametrizadas
Vistas del servidor	EJS	Formularios HTML con GET y POST (AA2-EV02)
Seguridad	bcrypt y jsonwebtoken	Contraseñas cifradas y sesiones con JWT (AA5)
Configuración	Archivo .env	Credenciales fuera del código
Base de datos	MySQL 8 y MySQL Workbench	Base de datos deportes creada en la Guía 6
Front-end	React con Vite	Interfaz de la tienda (AA4)
Calidad de código	ESLint y Prettier	Revisar y dar formato según los estándares (AA1-EV02)
Pruebas	Jest y Supertest	Pruebas automáticas de los endpoints
Editor	Visual Studio Code	Escritura de código y cliente Git integrado
Pruebas de API	Postman	Probar y documentar los endpoints (AA5)
Diseño	Figma	Prototipos y componentes de la interfaz

Nota: Antes de instalar, revisa en el sitio oficial de cada herramienta cuál es la versión estable vigente.

6. Selección de herramientas de versionamiento

6.1 Tipos de sistemas de control de versiones

Criterio	Centralizado (Subversion)	Distribuido (Git)
Dónde está el historial	Solo en el servidor	Copia completa en cada equipo
Trabajo sin internet	No permite hacer commits	Permite commits, ramas e historial
Ramas	Costosas; se usan poco	Rápidas y livianas; una por funcionalidad

Criterio	Centralizado (Subversion)	Distribuido (Git)
Si falla el servidor	Se puede perder el historial	Cualquier copia restaura el proyecto
Uso en la industria	Proyectos antiguos	Estándar actual

Se selecciona **Git** por ser distribuido, gratuito, de código abierto y el estándar de la industria. Es la herramienta del componente formativo «Integración continua» y la que piden las evidencias AA1-EV03 a AA1-EV05.

6.2 Plataformas para el repositorio remoto

Criterio	GitHub	GitLab	Bitbucket
Repositorios privados gratis	Sí, ilimitados	Sí, ilimitados	Sí, hasta 5 usuarios
Integración continua	GitHub Actions	GitLab CI/CD	Bitbucket Pipelines
Gestión de tareas	Issues y Projects	Issues y tableros	Integración con Jira
Beneficio educativo	GitHub Education	Programa GitLab for Education	Plan académico
Comunidad y portafolio	La más grande	Grande	Menor

Se selecciona **GitHub** porque su plan gratuito cubre el proyecto, integra en un solo lugar el código, las tareas (Issues y Projects) y la integración continua (Actions), y el repositorio sirve como portafolio profesional del aprendiz. GitLab es una alternativa equivalente si el instructor lo prefiere.

6.3 Herramientas seleccionadas

Herramienta	Función
Git (línea de comandos)	Control de versiones local: commits, ramas, historial
GitHub	Repositorio remoto, pull requests y copia en la nube
Cliente Git de VS Code	Ver cambios y hacer commits desde el editor
GitHub Projects e Issues	Tablero del sprint y seguimiento de historias de usuario
GitHub Actions	Integración continua: compilar y probar en cada push
Autenticación SSH o token personal	Conexión segura entre el equipo y GitHub

7. Estrategia de versionamiento

7.1 Organización del repositorio

Se usará un solo repositorio, **tienda-deportes**, con una carpeta por módulo. Así cada evidencia entrega el mismo enlace y el historial muestra la evolución completa del proyecto. El ejercicio de la AA1-EV05 se hace aparte, en el repositorio Programa-Git que pide la guía.

```
tienda-deportes/
├── database/              script deportes.sql (Guía 6)
├── backend/               Node.js + Express (AA2, AA3, AA5)
│   ├── src/
│   │   ├── config/        conexión a MySQL
│   │   ├── rutas/         rutas de Express
│   │   ├── controladores/ reciben la petición y responden
│   │   ├── servicios/     reglas del negocio
│   │   ├── modelos/       consultas SQL con mysql2
│   │   ├── middlewares/   errores y autenticación
│   │   └── vistas/        plantillas EJS (AA2-EV02)
│   ├── app.js
│   └── package.json
├── frontend/              React (AA4)
├── docs/                  informes, pruebas y colección de Postman
├── .github/workflows/ci.yml
├── .gitignore
└── README.md
```

7.2 Modelo de ramas

Se adapta Git Flow de forma simplificada:

- **main:** versión estable. Solo recibe cambios desde develop al cerrar cada actividad y se marca con una etiqueta.
- **develop:** integración del trabajo en curso.
- **feature/nombre:** una rama por historia de usuario o funcionalidad, por ejemplo feature/crud-productos.
- **fix/nombre:** corrección de errores, por ejemplo fix/validar-stock.

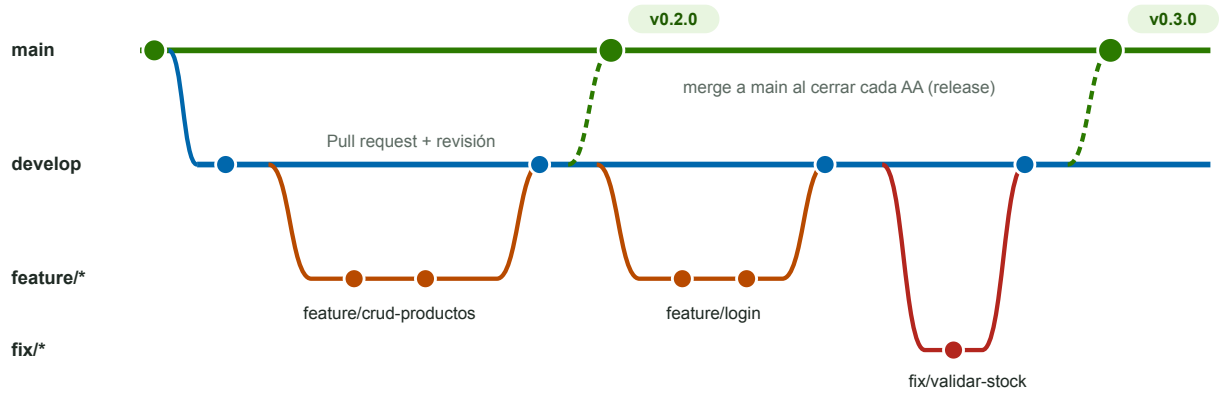


Figura 1. Modelo de ramas del proyecto.

Las ramas feature y fix se integran a develop con un **pull request**. main y develop se protegen en GitHub (Settings › Branches) para que no se pueda hacer push directo.

7.3 Convención de mensajes de commit

Se usa el formato **Conventional Commits**: tipo, dos puntos y una descripción corta en imperativo.

Tipo	Cuándo se usa	Ejemplo
feat	Nueva funcionalidad	feat: agregar CRUD de productos con mysql2
fix	Corrección de un error	fix: validar stock antes de crear el pedido
docs	Documentación	docs: actualizar README con la instalación
refactor	Mejora del código sin cambiar su función	refactor: mover las consultas a producto.modelo.js
test	Pruebas	test: probar el inicio de sesión con datos inválidos
chore	Configuración y tareas	chore: agregar .gitignore de Node

7.4 Versiones y etiquetas

Se aplica **versionado semántico** (MAYOR.MENOR.PARCHE). Cada actividad terminada sube la versión menor y se marca con una etiqueta: v0.1.0 (AA1), v0.2.0 (AA2), v0.3.0 (AA3), v0.4.0 (AA4) y v1.0.0 al completar las API (AA5).

7.5 Archivos de configuración

- **.gitignore:** excluye node_modules/, dist/, coverage/, *.log y .env.
- **README.md:** descripción, tecnologías, instalación, ejecución y autor.

- **.env y .env.example:** .env guarda el usuario y la contraseña de MySQL y no se sube; .env.example lista las variables sin valores para que se sepa cuáles configurar.

7.6 Flujo de trabajo diario

```
git switch develop
git pull origin develop
git switch -c feature/crud-productos
# ... programar y probar ...
git add .
git commit -m "feat: agregar CRUD de productos con mysql2"
git push -u origin feature/crud-productos
# En GitHub: abrir el pull request hacia develop y fusionarlo
```

8. Integración continua

Con **GitHub Actions** cada push o pull request revisa el estilo y ejecuta las pruebas del back-end, y compila el front-end. Si algo falla, GitHub lo marca en rojo antes de integrar el cambio. El archivo .github/workflows/ci.yml queda así:

```yaml
name: CI
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]
jobs:
  backend:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 24
          cache: npm
          cache-dependency-path: backend/package-lock.json
      - run: npm ci
      - run: npm run lint
      - run: npm test
  frontend:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: frontend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 24
          cache: npm
          cache-dependency-path: frontend/package-lock.json
      - run: npm ci
      - run: npm run build
```

Nota: El workflow funciona cuando existan las carpetas backend y frontend con su package.json (AA2 y AA4). En la AA1 basta con crear el archivo.

9. Buenas prácticas de seguridad

- Proteger las ramas main y develop y exigir pull request.
- Autenticarse con llave SSH o token personal con permisos mínimos; nunca con la contraseña de la cuenta.
- No subir contraseñas, tokens ni archivos .env; si se sube uno por error, cambiarlo de inmediato.
- Activar la verificación en dos pasos de la cuenta de GitHub.
- Hacer commits pequeños y frecuentes, y push al terminar cada sesión de trabajo.

10. Conclusiones

El plan organiza las 498 horas de la guía en sprints con un entregable claro por actividad, y relaciona cada módulo con la tecnología que lo construye. Usar Node.js con Express en el back-end y React en el front-end deja todo el proyecto en JavaScript, con un equivalente claro para cada tecnología que propone la guía. Git y GitHub cubren el control de versiones local y remoto, la gestión de tareas y la integración continua en un solo ecosistema gratuito.

Definir desde el inicio la estructura del repositorio, el modelo de ramas y las convenciones de commits evita desorden en el historial y prepara el trabajo de las siguientes evidencias, empezando por los estándares de codificación de la AA1-EV02.

11. Referencias

Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2

GitHub. (2026). GitHub Docs: Acerca de GitHub Actions. https://docs.github.com/es/actions

Conventional Commits. (2019). Conventional Commits 1.0.0. https://www.conventionalcommits.org/es/v1.0.0/

Preston-Werner, T. (2013). Versionado semántico 2.0.0. https://semver.org/lang/es/

Schwaber, K. y Sutherland, J. (2020). La Guía de Scrum. https://scrumguides.org/

SENA. (2024). Guía de aprendizaje 7: Codificar los módulos del software. Programa ADSO 228118.